

재귀호출로 이해하는 완전탐색과 순열, 그리고 Brute Force 적용

- 🔗 글의 목표: 재귀호출이 왜 필요한지부터 시작해, **중복순열·순열·완전탐색**으로 이어지는 흐름을 자연스럽게 정리한다.
- 🔑 핵심 키워드: **재귀호출**, **기저조건**, **Level**, **Branch**, **path**, **used/visited**, **순열**, **중복순열**, **Brute Force**, **가지치기**
- ★ 중요도: 상
- 📖 한눈에 보는 내용:
이번 강의는 단순히 “재귀 함수를 어떻게 쓰는가”를 넘어서, **재귀를 이용해 N중 반복을 표현하고**, 그 구조를 이용해 **모든 경우를 탐색하는 완전탐색 문제**를 푸는 흐름까지 연결한다.
초반에는 반복과 재귀의 차이, 함수 호출의 동작 원리, 기저조건的重要性을 익히고, 이후에는 **중복순열과 순열 생성**, 마지막에는 **주사위**, **트럼프 카드**, **Baby-gin** 같은 문제에 적용하는 방식으로 확장된다.
- 🔗 관련 문제 / 주제(있다면): **순열/조합 생성**, **DFS적 사고**, **백트래킹**, **완전탐색**, **가지치기**

1. 들어가며

이 글에서는 **재귀호출을 이용해 반복 구조를 일반화하는 방법**과, 그 아이디어가 **완전탐색 문제 해결**로 어떻게 이어지는지를 정리한다.

처음에는 2중 for문처럼 단순한 반복 예시로 시작하지만, 강의를 진행될수록 핵심은 분명해진다.

“반복문으로는 고정된 깊이만 쉽게 표현할 수 있지만, 재귀는 깊이가 바뀌는 탐색 구조를 자연스럽게 만들 수 있다.”

이 차이를 이해하면 순열, 중복순열, 주사위 합 문제처럼 **모든 경우를 하나씩 만들어 보는 문제**를 훨씬 체계적으로 다룰 수 있다.

즉, 이번 강의는 재귀 문법을 익히는 시간이 아니라, **탐색 문제를 바라보는 사고방식**을 만드는 시간이라고 볼 수 있다.

2. 핵심 개념 정리

이 섹션에서는 강의 전체를 이해하는 데 필요한 핵심 개념을 먼저 정리한다.

2-1. 반복과 재귀는 비슷하지만, 표현력이 다르다

반복(**for**, **while**)과 재귀(**함수가 자기 자신을 다시 호출**)은 비슷한 작업을 수행할 수 있다. 하지만 둘의 강점은 다르다.

구분	반복	재귀
기본 방식	같은 코드를 여러 번 실행	더 작은 같은 문제를 다시 호출
강점	단순 반복 처리에 강함	트리 구조, 경우의 수 생성에 강함
익숙한 예	2중 for문, 3중 for문	순열, 부분집합, DFS, 백트래킹

구분	반복	재귀
주의점	중첩 깊이가 고정돼야 편함	기저조건이 없으면 무한호출 위험

💡 핵심

재귀는 “어렵게 반복을 쓰는 문법”이 아니라,
반복 깊이가 유동적인 문제를 더 자연스럽게 표현하는 도구다.

2-2. 재귀에서 가장 중요한 것은 기저조건이다

재귀는 자기 자신을 계속 호출하므로, 반드시 멈추는 조건이 있어야 한다.
이 멈추는 조건을 **기저조건(base case)** 이라고 한다.

예를 들어 숫자 0부터 10까지 출력하는 재귀 함수라면,
`num > 10`일 때 종료해야 무한호출이 발생하지 않는다.

⚠ 주의

재귀를 처음 배울 때 가장 많이 하는 실수는 두 가지다.

1. 기저조건을 빼먹는 것
2. 호출 전/후 상태 복구를 하지 않는 것

특히 path를 사용하는 탐색 문제에서는 `append()` 후 반드시 `pop()`이 따라와야 한다.

2-3. 재귀를 이해하려면 함수 호출 구조를 알아야 한다

강의에서는 재귀를 본격적으로 다루기 전에, 일반 함수 호출의 특징부터 짚는다.

특징 1) 정수 같은 기본값은 “값”이 복사되어 전달된다

함수에 `int` 값을 넘기면, 원본 변수 자체가 넘어가는 것이 아니라 **값이 복사된 새로운 지역 변수**가 사용된다.

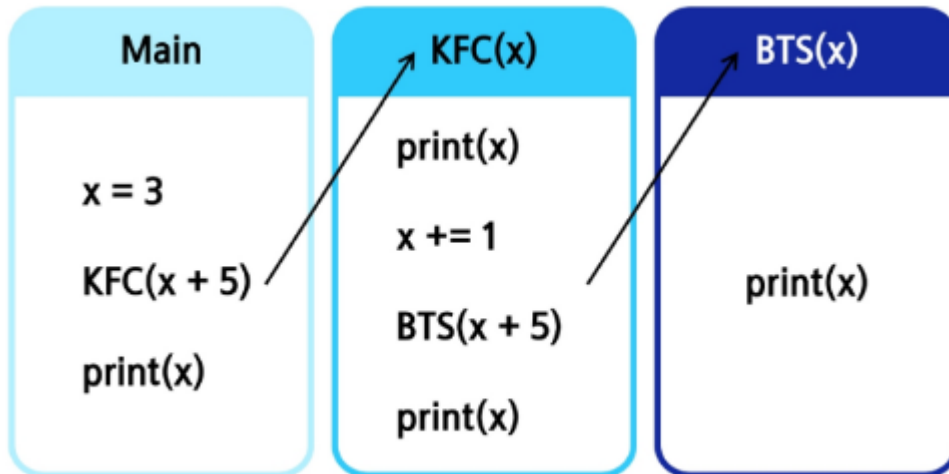


〈그림1 알고리즘 응용_원전탐색_재귀를 연습하기 전, 알아야 할 함수의 특징〉

이 그림은 `main`의 `x`와 함수 내부의 `x`가 같은 이름이어도 서로 다른 변수라는 점을 보여준다.
그래서 함수 안에서 `x += 1`을 해도 바깥의 `x`는 그대로 유지된다.

특징 2) 함수가 끝나면 “호출한 자리”로 돌아온다

함수가 끝나면 무조건 main으로 돌아가는 것이 아니라,
그 함수를 호출했던 정확한 위치로 돌아간다.



<그림2_알고리즘 응용_완전탐색_재귀를 연습하기 전, 알아야 할 함수의 특징 >

이 감각은 재귀를 이해할 때 매우 중요하다.
재귀는 사실 "새로운 함수 호출이 계속 쌓였다가, 끝에서부터 되돌아오는 구조"이기 때문이다.

🧠 기억할 것

재귀는 위에서 아래로만 내려가는 과정이 아니라,
내려가고 돌아오고를 반복하는 흐름까지 포함해서 이해해야 한다.

2-4. Level, Branch, path는 재귀 탐색의 핵심 언어다

강의 전체를 관통하는 핵심 표현은 아래 세 가지다.

- **Level:** 지금 몇 번째 선택까지 진행했는가
- **Branch:** 현재 단계에서 몇 개의 선택지가 있는가
- **path:** 지금까지 무엇을 선택해 왔는가

예를 들어 주사위를 3번 던진다면,

- Level = 3
- Branch = 6
- path = 지금까지 선택한 주사위 눈

이렇게 볼 수 있다.

이 관점을 익히면 문제를 보면 바로 이렇게 바꿔 말할 수 있다.

"아, 이 문제는 Level이 몇이고, 각 Level마다 Branch가 몇 개인지부터 정리하면 되겠구나."

3. 본문 정리

이 섹션에서는 강의 흐름에 맞춰 개념이 어떻게 확장되는지 정리한다.

3-1. 2중 for문에서 N중 for문으로: 왜 재귀가 필요한가

가장 단순한 예시는 1, 2, 3으로 만들 수 있는 두 자릿수 출력이다.

```
for a in range(1, 4):
    for b in range(1, 4):
        print(a, b)
```

이 코드는 2중 for문이라 간단하다.

하지만 자릿수가 3, 4, 5처럼 늘어나면 어떻게 될까?

- N이 2면 2중 for문
- N이 3이면 3중 for문
- N이 9면 9중 for문

이 방식은 너무 비효율적이다.

바로 이 지점에서 재귀가 등장한다.

3-2. 재귀는 N중 for문을 일반화한다

재귀를 쓰면 "현재 자리에서 1, 2, 3 중 하나를 고르고 다음 자리로 이동"이라는 패턴을 공통으로 표현할 수 있다.

```
path = []
N = 3

def run(lev):
    if lev == N:
        print(path)
        return

    for i in range(1, 4):
        path.append(i)
        run(lev + 1)
        path.pop()

run(0)
```

이 코드의 핵심은 단순하다.

1. 현재 자리에 숫자를 하나 넣는다.
2. 다음 자리로 재귀호출한다.
3. 돌아오면 방금 넣은 숫자를 빼고 다른 선택을 시도한다.

즉, **선택** → **내려감** → **복구**가 재귀 탐색의 기본 패턴이다.

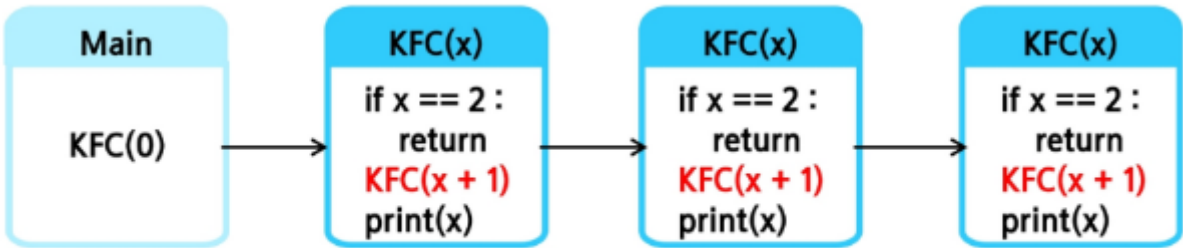
💡 포인트

여기서 `path.pop()`은 선택을 "취소"하는 동작이다.

이 한 줄이 있어야 이전 상태로 돌아가 다른 경우를 이어서 만들 수 있다.

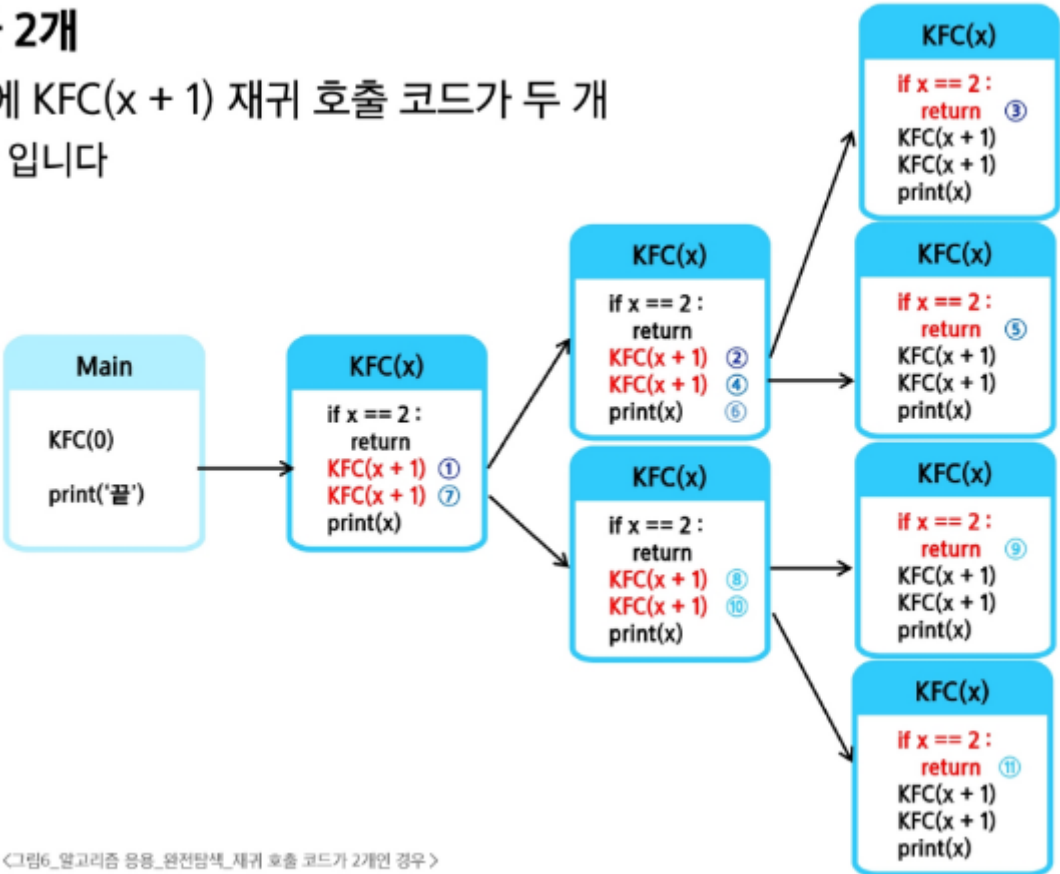
3-3. 재귀호출이 여러 개면, 트리처럼 퍼진다

강의에서는 재귀호출이 1개, 2개, 3개일 때 호출 구조가 어떻게 달라지는지를 그림으로 보여준다.

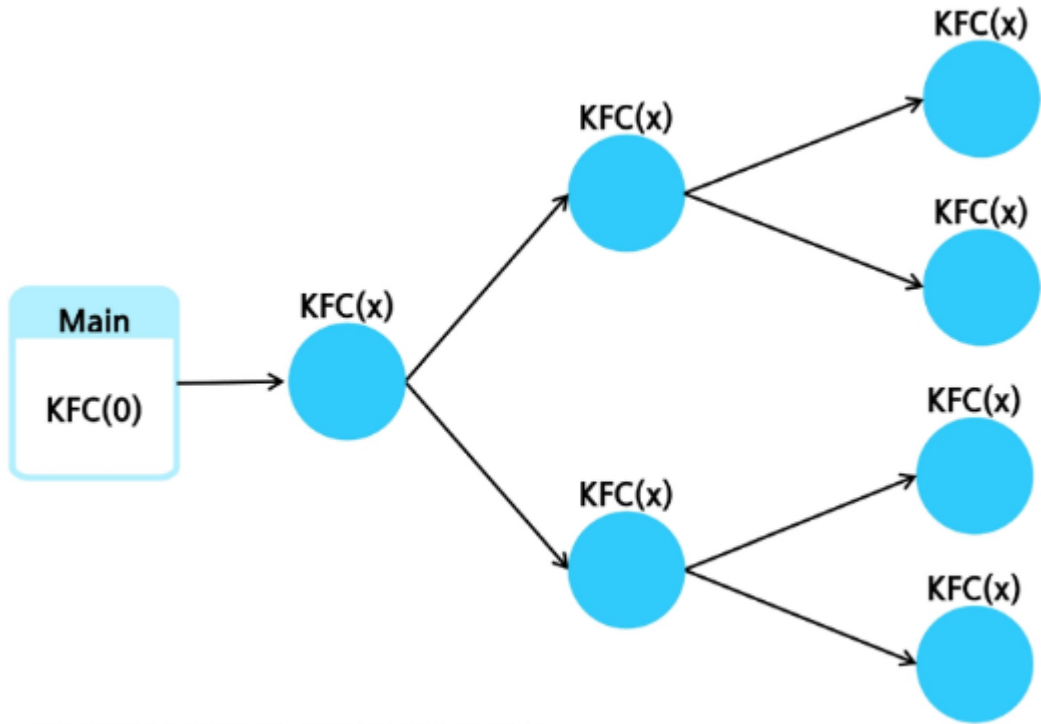


<그림5_알고리즘 응용_원전탐색_재귀 호출 코드가 1개인 경우>

1 2개
에 KFC(x + 1) 재귀 호출 코드가 두 개
)입니다



<그림6_알고리즘 응용_원전탐색_재귀 호출 코드가 2개인 경우>



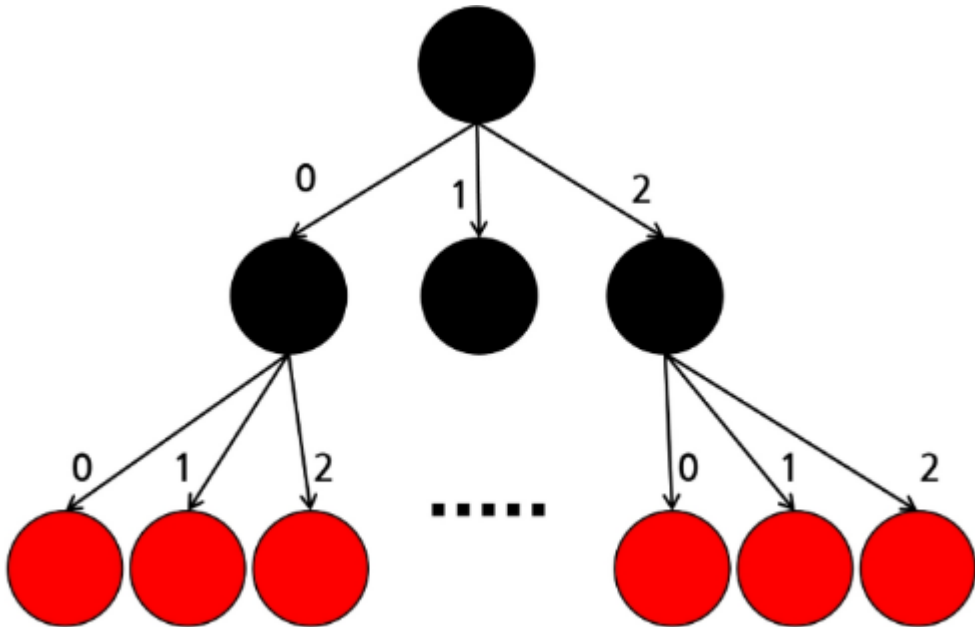
<그림7_알고리즘 응용_완전탐색_재귀호출 코드가 2개, 간략화 하여 그리기 1>

이 그림들이 말하는 핵심은 하나다.
재귀는 코드를 위에서 아래로 읽는 것이 아니라, **호출 트리 전체를 머릿속에 그릴 수 있어야** 한다는 점이다.
재귀호출이 2개면 각 정점에서 두 갈래로, 3개면 세 갈래로 뻗는다.
그래서 완전탐색 문제는 자연스럽게 **트리 탐색 문제처럼 해석**할 수 있다.

3-4. 중복순열: 같은 숫자를 여러 번 써도 되는 경우

강의에서는 카드 0, 1, 2 중에서 2장을 뽑는 예시로 **중복순열**을 설명한다.
중복순열은 다음과 같은 특징을 가진다.

- 순서를 고려한다.
- 같은 원소를 여러 번 사용할 수 있다.
- 따라서 [0, 0], [1, 1], [2, 2]도 허용된다.



<그림15_알고리즘 응용_완전탐색_중복 순열 구현 2>

핵심 아이디어는 간단하다.

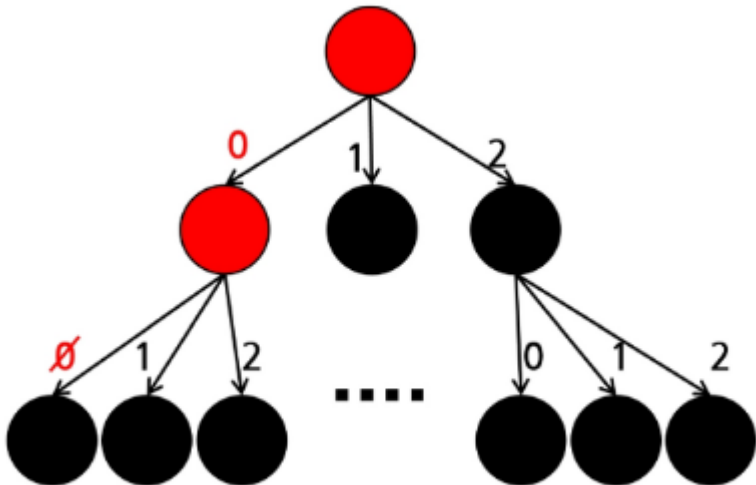
- 1. 재귀호출을 할 때마다 선택한 값을 **path**에 남긴다.
- 2. 마지막 Level에 도달하면 **path**를 출력한다.

즉, **path**는 지금까지 선택한 흔적이다.

3-5. 순열: 중복을 허용하지 않는 경우

이제 중복순열 코드에 “이미 쓴 숫자는 다시 쓰지 못하게 하는 장치”를 추가하면 순열이 된다.

그 장치가 바로 **used** 또는 **visited** 배열이다.



<그림24_알고리즘 응용_완전탐색_중복을 제거한 순열 생성 예시>

원리는 다음과 같다.

- 어떤 숫자를 고르기 전, 이미 사용했는지 확인한다.
- 사용하지 않았다면 **used[i] = True**로 표시한다.
- 재귀가 끝나고 돌아오면 **used[i] = False**로 원상복구한다.

이 패턴은 이후 DFS, 백트래킹, 그래프 탐색에서도 계속 등장한다.

💡 핵심

중복순열 + 사용 여부 체크 = 순열

3-6. 완전탐색은 “가능한 모든 경우를 다 해보는 것”이다

완전탐색(Brute Force)은 가능한 경우를 모두 시도해서 답을 찾는 방법이다.

겉보기에는 단순해 보이지만, 실전에서는 매우 중요하다.

- 아이디어 검증용으로 좋다.
- 작은 입력에서는 정답을 확실하게 찾을 수 있다.
- 가지치기까지 붙이면 생각보다 강력하다.

강의의 예시도 모두 같은 흐름으로 풀린다.

- 주사위 3개를 던져 합이 10 이하인 경우 세기
- 5장의 카드 중 같은 문자가 3장 연속 나오는 경우 세기
- 6장의 카드가 Baby-gin인지 검사하기

결국 질문은 항상 비슷하다.

“모든 경우를 어떻게 만들 것인가?”

“만든 경우가 조건을 만족하는지 어떻게 검사할 것인가?”

4. 적용 / 예시 / 코드 해설

이 섹션에서는 강의 내용을 실제 코드 흐름으로 연결한다.

4-1. 예시 1: 재귀로 N자리 수 만들기

이 코드는 1, 2, 3을 사용해 길이 N인 모든 수를 만드는 기본 구조다.

```
# 지금까지 선택한 숫자를 저장할 리스트
path = []

# 만들고 싶은 자릿수
N = 3

def run(lev):
    # 기저조건: N개를 모두 골랐다면 현재 경로를 출력하고 종료
    if lev == N:
        print(path)
        return

    # 현재 자리(lev)에는 1, 2, 3 중 하나를 넣을 수 있다
    for num in range(1, 4):
        # 1) 현재 선택을 기록
        path.append(num)
```

```

# 2) 다음 자리로 이동
run(lev + 1)

# 3) 돌아오면 현재 선택을 취소해서 다른 숫자도 시도
path.pop()

# 0번째 자리부터 시작
run(0)

```

코드 흐름 해설

- `lev`는 지금 몇 번째 자리까지 채웠는지를 의미한다.
- `path`는 지금까지 만든 숫자들의 기록이다.
- `lev == N`이 되면 `N`자리를 모두 채운 것이므로 출력한다.
- 각 단계에서 `1, 2, 3`을 하나씩 넣어 보며 탐색한다.

자주 하는 실수 / 디버깅 포인트

- `path.pop()`을 빼먹으면 이전 선택이 남아 경로가 꼬인다.
- `if lev == N`: 대신 `if lev > N`:처럼 어색하게 쓰면 종료 시점이 흔들릴 수 있다.
- `path`를 출력할 때 문자열처럼 보고 싶다면 `print(*path, sep='')`처럼 바꿀 수 있다.

4-2. 예시 2: 중복순열 만들기

이 코드는 `0, 1, 2` 중에서 2개를 골라 **중복을 허용하는 모든 경우**를 출력한다.

```

# 현재까지 고른 카드
path = []

def recur(cnt):
    # 기저조건: 카드 2장을 모두 골랐다면 출력
    if cnt == 2:
        print(path)
        return

    # 현재 자리에서 0, 1, 2 중 하나를 고를 수 있다
    for i in range(3):
        # 현재 선택 기록
        path.append(i)

        # 다음 자리 선택
        recur(cnt + 1)

        # 다른 경우를 위해 원상복구
        path.pop()

# 0장 고른 상태에서 시작
recur(0)

```

코드 흐름 해설

이 문제는 Branch가 3, Level이 2인 트리로 볼 수 있다.

- 첫 번째 자리: 0, 1, 2 중 하나 선택
- 두 번째 자리: 다시 0, 1, 2 중 하나 선택

그래서 결과는 총 $3 \times 3 = 9$ 개가 된다.

자주 하는 실수 / 디버깅 포인트

- 중복순열과 순열을 헷갈려 **used** 배열을 괜히 넣는 경우가 많다.
- 중복이 허용되는 문제에서는 같은 숫자를 다시 골라도 된다.
- 따라서 이 단계에서는 **used**가 필요 없다.

4-3. 예시 3: 순열 만들기

이번에는 같은 숫자를 두 번 쓰지 못하게 만들어 보자.

즉, **중복 없는 순열**이다.

```
# 현재까지 고른 숫자
path = []

# 숫자 0, 1, 2의 사용 여부를 저장
used = [False, False, False]

def perm(lev):
    # 기저조건: 2개를 다 골랐다면 출력
    if lev == 2:
        print(*path)
        return

    # 0, 1, 2를 차례로 확인
    for i in range(3):
        # 이미 사용한 숫자라면 건너뛰다
        if used[i]:
            continue

        # 사용하지 않은 숫자라면 선택 처리
        used[i] = True
        path.append(i)

        # 다음 자리로 이동
        perm(lev + 1)

        # 탐색을 마치고 돌아오면 원상복구
        path.pop()
        used[i] = False

# 0번째 자리부터 시작
perm(0)
```

코드 흐름 해설

이 코드는 중복순열 코드와 거의 같지만, 딱 하나가 다르다.

- 선택하기 전에 `used[i]`를 확인한다.
- 선택했다면 `True`로 표시하고,
- 되돌아오면 다시 `False`로 바꾼다.

즉, 선택 여부를 상태로 저장하고 복구하는 구조가 핵심이다.

자주 하는 실수 / 디버깅 포인트

- `if used[i]: continue`를 빼먹으면 순열이 아니라 중복순열이 된다.
- `used[i] = False`를 빼먹으면 이후 탐색이 막혀 결과가 줄어든다.
- 필기 예시 중에는 `if not selected:`처럼 적힌 부분이 있는데, 실제 의도는 보통 `if not selected[j]:`처럼 **현재 원소가 사용되었는지 확인**하는 형태다.

⚠ 주의

재귀 예시 코드 중 일부는 강의용 요약 필기라서, 실제 제출 코드로는 인덱스 조건과 변수명이 조금 더 정확해야 한다.

4-4. 예시 4: 주사위 눈의 합이 10 이하인 경우 세기

이 예시는 완전탐색에 **가지치기**가 처음으로 붙는 대표 문제다.

문제:

- 주사위를 3번 던진다.
- 합이 10 이하인 경우의 수를 센다.

```
# 조건을 만족하는 경우의 수를 저장
cnt = 0

def search(lev, total):
    global cnt

    # 가지치기: 이미 합이 10을 넘었다면 더 볼 필요가 없다
    if total > 10:
        return

    # 기저조건: 주사위 3개를 모두 골랐다면 경우의 수 1 증가
    if lev == 3:
        cnt += 1
        return

    # 현재 주사위 눈은 1~6 중 하나
    for face in range(1, 7):
        # 다음 주사위를 선택하면서 누적합도 함께 전달
```

```

        search(lev + 1, total + face)

# 아직 아무것도 고르지 않은 상태에서 시작
search(0, 0)

print(cnt) # 108

```

코드 흐름 해설

이 문제는 단순히 `path`를 출력하는 데서 끝나지 않고,
탐색 중간에 조건을 검사해 더 이상 의미 없는 경로를 중단한다는 점이 중요하다.

- Level = 3
- Branch = 6
- 상태 = 현재까지의 합 `total`

여기서 `total > 10`이면 남은 주사위를 어떻게 골라도 조건을 만족할 수 없으므로 즉시 종료한다.

이것이 바로 가지치기(**pruning**)이다.

자주 하는 실수 / 디버깅 포인트

- 합이 10을 넘었는데도 끝까지 내려가면 불필요한 탐색이 많아진다.
- `cnt`를 세는 위치는 `lev == 3`인 지점이어야 한다.
- 탐색용 문제에서는 "출력"과 "카운팅"을 섞지 않도록 목적을 분리하는 것이 좋다.

💡 핵심

완전탐색은 무조건 느린 것이 아니라,
조건을 이용해 가지치기하면 훨씬 효율적으로 바꿀 수 있다.

4-5. 예시 5: 연속 3장의 같은 트럼프 카드 검사

이 문제는 "모든 경우 생성"과 "조건 검사 함수 분리"를 함께 보여준다.

```

# 사용할 카드 종류
cards = ['A', 'J', 'Q', 'K']

# 현재까지 뽑은 카드
path = []

# 조건을 만족하는 경우의 수
cnt = 0

def has_cont_three():
    # 길이 5의 path 안에서 연속한 3장이 같은지만 확인
    if path[0] == path[1] == path[2]:
        return True
    if path[1] == path[2] == path[3]:
        return True

```

```

    if path[2] == path[3] == path[4]:
        return True
    return False

def recur(lev):
    global cnt

    # 카드 5장을 다 뽑았다면 조건 검사
    if lev == 5:
        if has_cont_three():
            cnt += 1
        return

    # 각 자리에는 4종류 카드 중 하나를 놓을 수 있다
    for i in range(4):
        path.append(cards[i])
        recur(lev + 1)
        path.pop()

recur(0)
print(cnt)

```

코드 흐름 해설

이 문제의 좋은 점은 역할 분리가 분명하다는 것이다.

- `recur()`는 모든 경우를 생성
- `has_cont_three()`는 완성된 경우가 조건을 만족하는지 검사

이렇게 나누면 코드가 훨씬 읽기 쉬워진다.

자주 하는 실수 / 디버깅 포인트

- 조건 검사 함수를 재귀 안에 억지로 섞으면 흐름이 복잡해진다.
- 먼저 "경우 생성"과 "검사"를 분리해서 생각하면 구조가 훨씬 깔끔해진다.

4-6. Baby-gin 문제는 왜 중요한가

강의 마지막의 Baby-gin 문제는 아직 코드보다 **문제 해석 방식**이 더 중요하다.

문제의 핵심은 아래와 같다.

- 6장의 카드를 어떤 순서로든 배치해 본다.
- 앞의 3장과 뒤의 3장이 각각
 - **run**(연속된 수) 이거나
 - **triplet**(같은 수 3장) 이면
- Baby-gin이다.

즉, 이 문제는 결국 이렇게 바뀐다.

1. 6장의 카드에 대한 모든 순열 생성

2. 각 순열에 대해 앞 3장 / 뒤 3장 검사
3. 하나라도 조건을 만족하면 정답

💡 포인트

이 문제는 완전탐색의 전형적인 사고 흐름을 잘 보여준다.

- 경우의 수를 만든다
- 조건을 검사한다
- 참인 경우가 하나라도 있으면 멈춘다

이후에는 이 구조가 백트래킹, 조합, 부분집합, DFS 문제로 자연스럽게 이어진다.

5. 배운 점 / 느낀 점 / 확장 포인트

이 강의를 통해 가장 크게 배울 수 있는 점은,
재귀를 “문법”이 아니라 **탐색 구조를 설계하는 도구**로 봐야 한다는 것이다.

처음에는 재귀가 단순히 자기 자신을 다시 호출하는 낯선 문법처럼 보이지만,
순열과 완전탐색 문제까지 연결해 보면 역할이 훨씬 분명해진다.

- 반복문은 고정된 깊이의 구조에 강하다.
- 재귀는 깊이가 커지거나 구조가 트리처럼 퍼지는 문제에 강하다.
- **path, used, lev** 같은 상태를 관리하면 경우의 수 생성이 체계화된다.
- 여기에 조건 검사를 붙이면 완전탐색 문제가 된다.
- 조건이 안 되는 경로를 미리 끊으면 가지치기가 된다.

실제로 이 흐름은 코딩테스트에서 아주 자주 등장한다.

- 순열 / 조합 / 부분집합 생성
- DFS / 백트래킹
- 게임판 탐색
- 특정 조건을 만족하는 경우의 수 세기

🔄 확장 포인트

1. **부분집합** 생성도 같은 구조로 만들 수 있다.
각 원소를 “고른다 / 안 고른다” 두 갈래로 나누면 된다.
2. **조합**은 순열과 달리 순서를 고려하지 않으므로,
다음 재귀호출의 시작 인덱스를 조절하는 방식으로 확장된다.
3. **백트래킹**은 완전탐색에 가지치기를 더한 형태로 볼 수 있다.
이번 강의의 주사위 합 예시가 그 출발점이다.

6. 요약 정리

이 섹션에서는 강의 핵심을 다시 짧게 정리한다.

📌 핵심

- 재귀는 더 작은 같은 문제를 호출하며 답을 구하는 방식이다.
- 재귀에는 반드시 **기저조건**이 필요하다.
- 재귀 탐색은 보통 **선택** → **재귀호출** → **복구** 패턴으로 작성한다.
- **path**는 지금까지의 선택 기록이고, **used**는 중복 방지 장치다.
- 중복순열은 같은 원소를 여러 번 써도 되고, 순열은 안 된다.
- 완전탐색은 가능한 모든 경우를 시도하는 방법이다.
- 가지치기를 하면 불필요한 경우를 조기에 제거할 수 있다.

☹ 기억할 것

- “이 문제의 Level은 몇인가?”
- “각 Level에서 Branch는 몇 개인가?”
- “지금까지 선택한 상태를 무엇으로 저장할 것인가?”
- “언제 멈추고, 언제 복구할 것인가?”

이 네 가지 질문이 재귀/완전탐색 문제를 푸는 기본 출발점이다.

7. 미니 퀴즈 또는 체크리스트

복습 질문

1. 반복문으로 표현하기 어려운 구조를 재귀가 더 잘 다루는 이유는 무엇일까?
2. 재귀 함수에서 기저조건이 없는 경우 어떤 문제가 생길까?
3. 중복순열과 순열의 차이는 무엇이며, 코드에서는 어떤 장치로 구분할까?
4. `path.append()` 뒤에 `path.pop()`이 필요한 이유는 무엇일까?
5. 완전탐색에서 가지치기는 어떤 상황에서 효과적일까?

이해 점검 체크리스트

- ☐ 재귀를 호출 트리 관점에서 설명할 수 있다.
- ☐ **Level**, **Branch**, **path**의 의미를 알고 있다.
- ☐ 중복순열 코드와 순열 코드의 차이를 설명할 수 있다.
- ☐ **used/visited** 배열이 왜 필요한지 알고 있다.
- ☐ 완전탐색과 가지치기의 차이를 이해하고 있다.